

Trees

1. Write an algorithm for counting the number of left external nodes in a binary tree, using the Binary Tree ADT.

left external node $\rightarrow$ no children, is a left child

$\therefore$ recursion works well

Plan:

- traverse whole tree, for each node, inspect left child
- if left child exists and is external, count it
- else, traverse both left and right subtrees recursively
- always recurse into right subtree

Code: (assuming we have `isExternal`, using pointers to node directly)

```
int countLeftLeaf(TreeNode* root) {
    if (root == nullptr) {
        return 0;
    }
    int count = 0;
    if (root->left != nullptr) {
        if (isExternal(root->left)) {
            count += 1;
        } else {
            count += countLeftLeaf(root->left);
        }
    }
    count += countLeftLeaf(root->right);
    return count;
}
```

$O(N)$ time complexity (we look at each node)

$O(H)$ space complexity (call stack) (can be $O(N)$)

2. How do you delete a node with two children in a binary search tree?

You must choose a successor, and you have two options:

in-order predecessor (largest in left subtree)

in-order successor (smallest in right subtree)

it takes the parent's place, and the parent is removed.

3. perfect binary tree: both full and complete
 full / proper binary tree: all nodes have 0 or 2 children
 complete binary tree: all levels fully filled except maybe the last one
 balanced binary tree: height difference between left & right subtrees is max 1.
4. Rule for Binary search Tree: For every node, the left subtree contains only elements smaller than the node, and the right greater.
5. Rule for AVL Tree: Self-balancing BST where height difference is at most 1 for any node between its left & right subtrees.
6. Draw a binary tree T such that
 - a) each internal node stores one character
 - b) preorder traversal yields EXAMFUN
 - c) inorder traversal yields MAXUEN

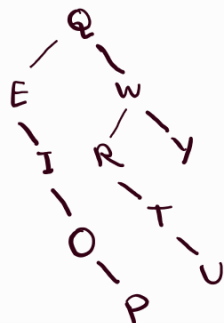


7. Design and write an algorithm to traverse and output the post order traversal of a binary tree.

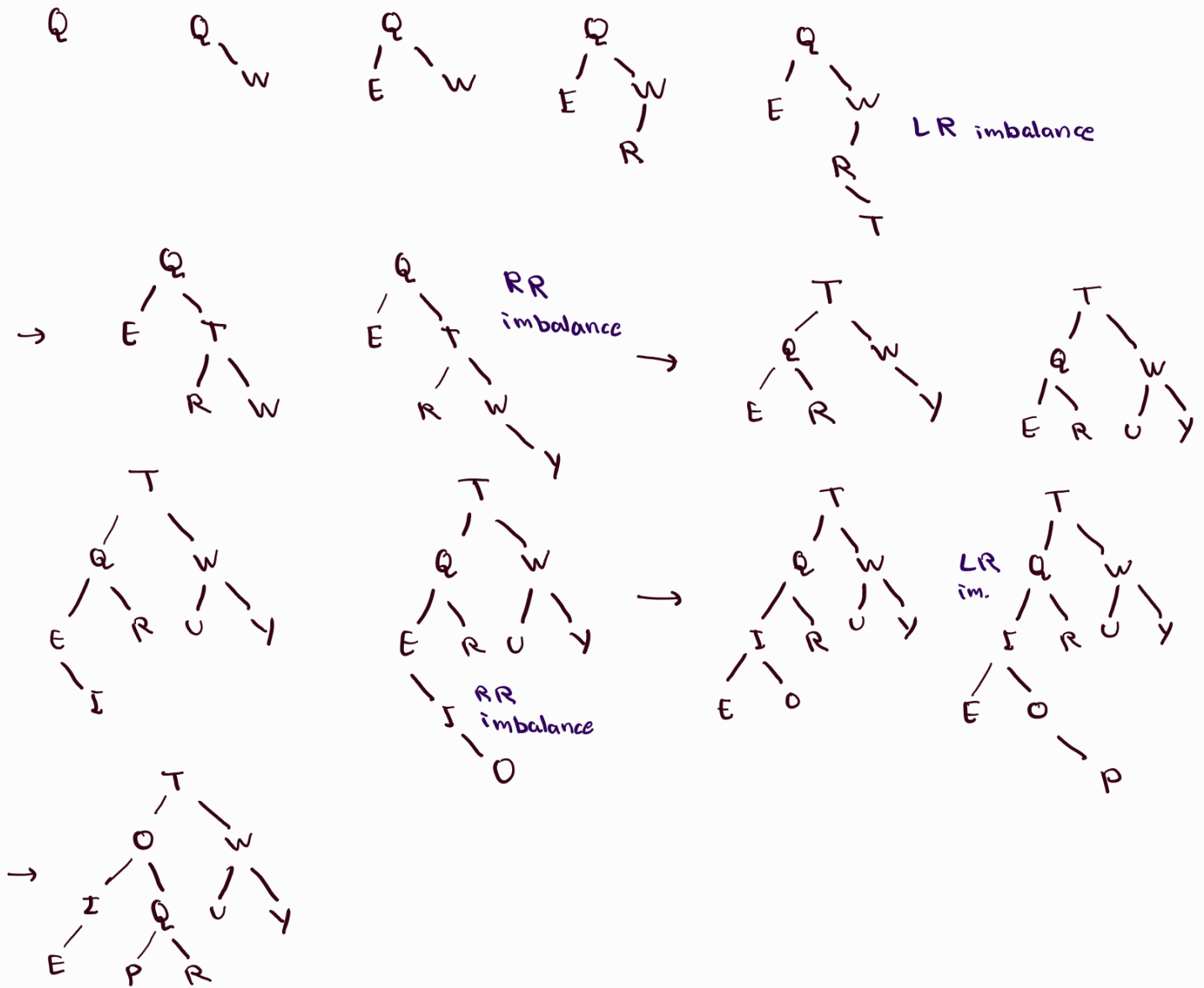
```

void postOrderPrint(TreeNode* root) {
    if (root == nullptr) {           ← base case
        return;
    }
    postOrderPrint(root->left); ← recurse left
    postOrderPrint(root->right); ← recurse right
    cout << root->val << " "; ← output current
}
  
```

8. Create a binary search tree with the input QWERTYUIOP



9. Insert QWERTYUIOP into an initially empty AVL Tree

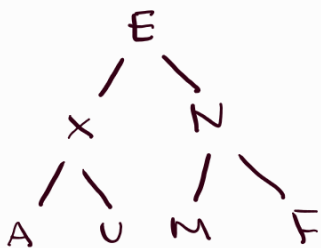


10. Represent the following array as a drawn tree, then convert it to a binary search tree, assuming you add elements to an initially empty BST by level order traversal of the given tree. What if you did a pre order traversal?

0 1 2 3 4 5 6

E X N A U M F

Tree



Level order $\rightarrow [E, X, N, A, U, M, F]$

Pre order $\rightarrow [E, X, A, U, N, M, F]$

Level order BST

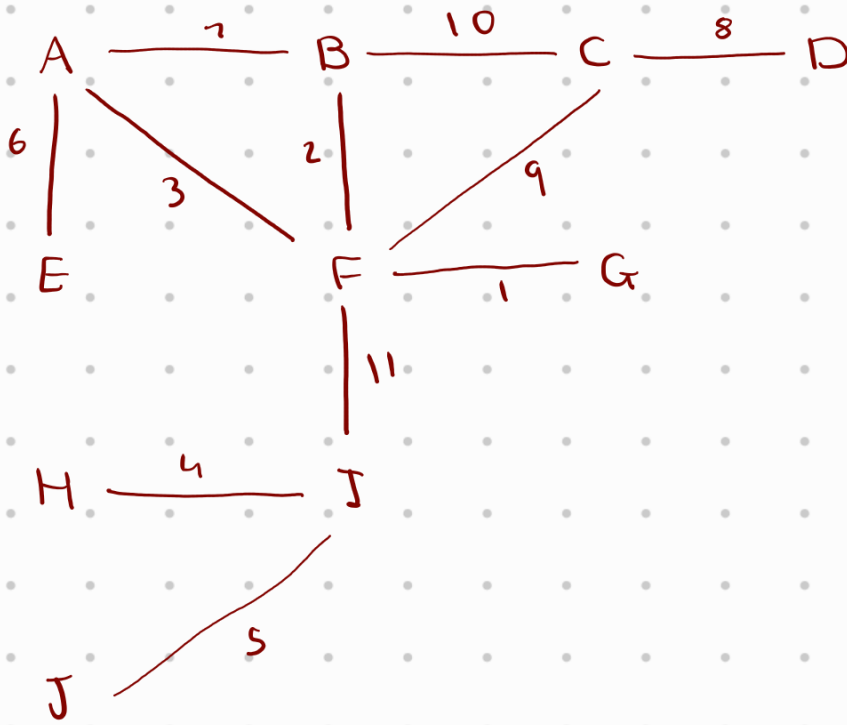


Pre order BST



Graphs

1. Graph Traversal (take smallest edge when applicable)



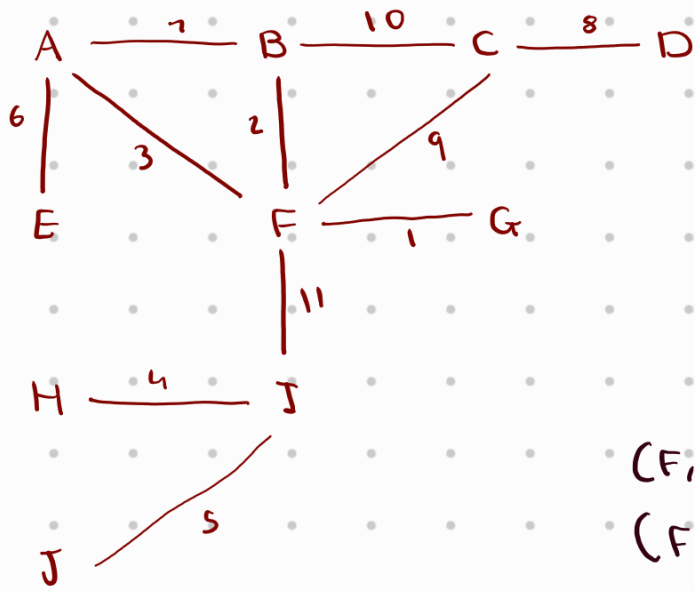
a) Starting at node A, output the breadth first traversal:

$A \rightarrow F \rightarrow E \rightarrow B \rightarrow G \rightarrow C \rightarrow J \rightarrow D \rightarrow H$

b) Starting at node B, output the depth first traversal:

$B \rightarrow F \rightarrow G \rightarrow A \rightarrow E \rightarrow C \rightarrow D \rightarrow J \rightarrow H$

2. Graph Algorithms



(a) Prim's Algorithm

starting at F, what is the MST formed by Prim's? (show edges in the order they're added to the MST)

(F, G, 1)

(F, B, 2)

(F, A, 3)

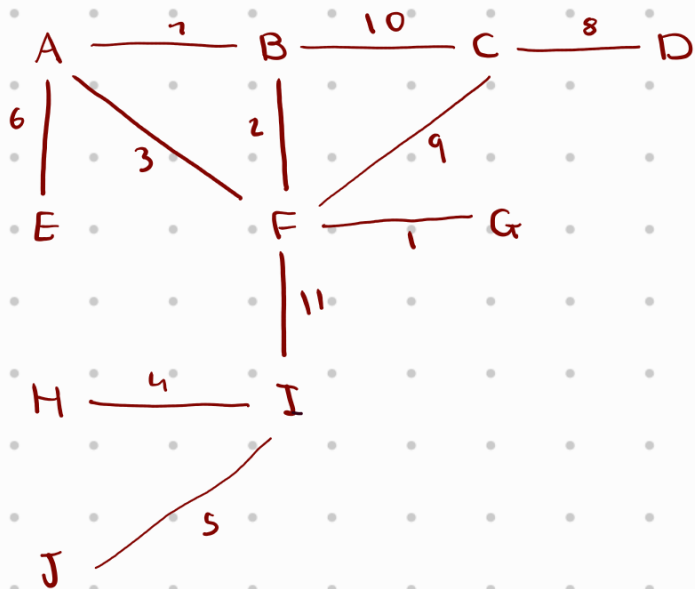
(A, E, 6)

(C, F, 9)

(C, D, 8)

(F, I, 10)

(I, H, 4)



(b) Kruskal's Algorithm

What is the MST formed by Kruskal's algorithm? Show edges in the order they're added.

(F, G, 1)

(B, F, 2)

(A, F, 3)

(H, I, 4)

(I, J, 5)

(A, E, 6)

(C, D, 8)

(C, F, 9)

3. Graph Terms

a) Incident: Edge to vertex if vertex is one of the edge's endpoints.

b) Adjacent: Vertex A and Vertex B if connected by an edge.

c) Degree: Total number of edges to vertex

i) in-degree: directed edges into

ii) out-degree: directed edges out of

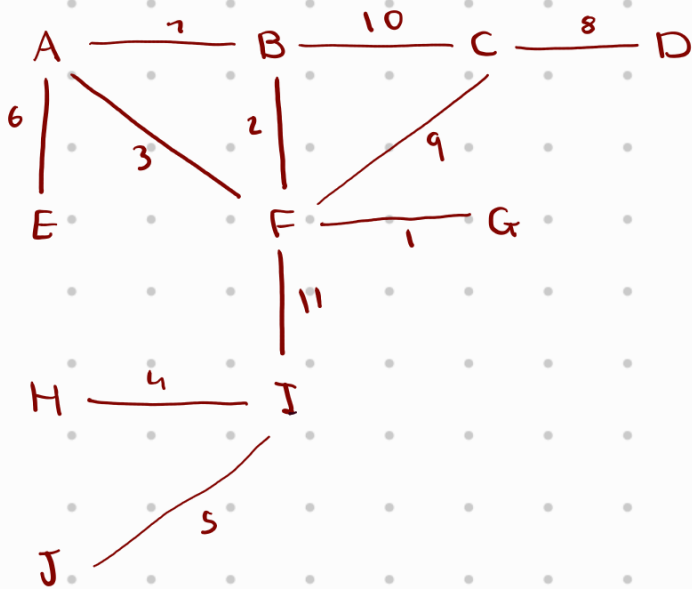
d) self loop: edge connecting vertex to itself

e) Parallel / multiple edges: two/more edges with same endpoints

f) spanning tree vs. spanning subgraph:

graph containing all vertices and spanning
subgraph without cycles + connected

4. Graph Representation



a) Represent the above graph as an adjacency list. Entries can be (adjacent vertex, edge weight).

A:

B:

C:

D:

E:

F:

G:

H:

I:

J:

Hashes

1. Add the values given, in order, to a hash table of capacity 11 using open addressing, using the hash function and collision handling method providing.

25 14 3 36 12 47

a) $h(x) = x \% 11$, linear probing

0	
1	12
2	
3	25
4	14
5	3
6	36
7	47
8	
9	
10	

b) $h(x) = x \% 11$, quadratic probing

0	
1	36
2	12
3	25
4	14
5	
6	
7	3
8	47
9	
10	

c) $h(x) = x \% 11$, double hashing

$$h_2(x) = 7 - (x \% 7)$$

0	
1	12
2	
3	25
4	
5	47
6	
7	
8	
9	36
10	14

2. a) Primary clustering: increased probe search times, long contiguous blocks of occupied slots, mainly affects linear probing.

b) Secondary clustering: Keys hash to same initial location follow same probe sequence, affects quadratic probing.

c) Load Factor: $\alpha = \frac{n}{m}$, n = stored elements
 m = total slots

3. Insert the following into an empty hash table using separate chaining:

25 14 3 36 12 47 58 8

using the hash function:

$$h(x) = x \% 11$$

0	1	2	3	4	5	6	7	8	9	10
	12		14					8		
			25							
			36							
			47							
			58							

Sorting note: stable means it preserves the relative order of equal/identical elements in the array.

1. Sort the following array in alphabetical order using merge sort. What are the time and space complexities? Is it stable? Show your steps.

["sit", "bit", "cat", "net", "dog", "map", "hop"]

[sit, bit, cat, net] [dog, map, hop]

[sit, bit] [cat, net] [dog, map] [hop]

[sit] [bit] [cat] [net] [dog] [map] [hop]

[bit, sit] [cat, net] [dog, map] [hop]

[bit, cat, net, sit] [dog, hop, map]

[bit, cat, dog, hop, map, net, sit]

Time: $O(N \log N)$

Space: $O(N)$

stable

2. Sort the following array in alphabetical order using merge sort. What are the time and space complexities? Is it stable? Show your steps, and use the last element as the pivot

["sit", "bit", "cat", "net", "dog", "map", "hop"]

| > < < > < > | pivot

bit sit

cat sit

bit cat dog hot sit map net
| < < | pivot | > < | pivot

map sit

bit cat dog hot
| < | pivot

map net sit
pivot pivot

bit cat dog hot map net sit